DEPLOYMENT WALKTHROUGH

# LMCache on CXL Memory

How the KV cache is placed in CXL memory, grown while serving, and shared between two servers. Shown step by step.

The basics | Deployment, step by step | Measured on this rig | Operating it | Upstream

LMCache multiprocess mode · dev branch · October 2026

AGENDA

# Today

### The basics

Why KV reuse matters. CXL memory, Device-DAX, MP mode, and the L1 and L2 tiers.

### Deployment, step by step

One server, the Device-DAX L1, hot-plug, the DAX L2, then two servers on one shared L1.

### Measured on this rig

TTFT by cache setup, CXL against the local SSD, bandwidth over time.

### Operating it

Failure handling, NUMA placement, /status and /metrics, common errors.

### Upstream

What has been merged, what is open, where the numbers come from.

### No typing today

Every step is shown as an animation. Scripts and raw results are in the materials folder for later.

Builds are advanced with the arrow key. Each deployment slide adds one element at a time.

CHAPTER 1

# The basics

Memory on a link, the Linux door, and the two processes that use it.

THE BASICS

# One document, two questions

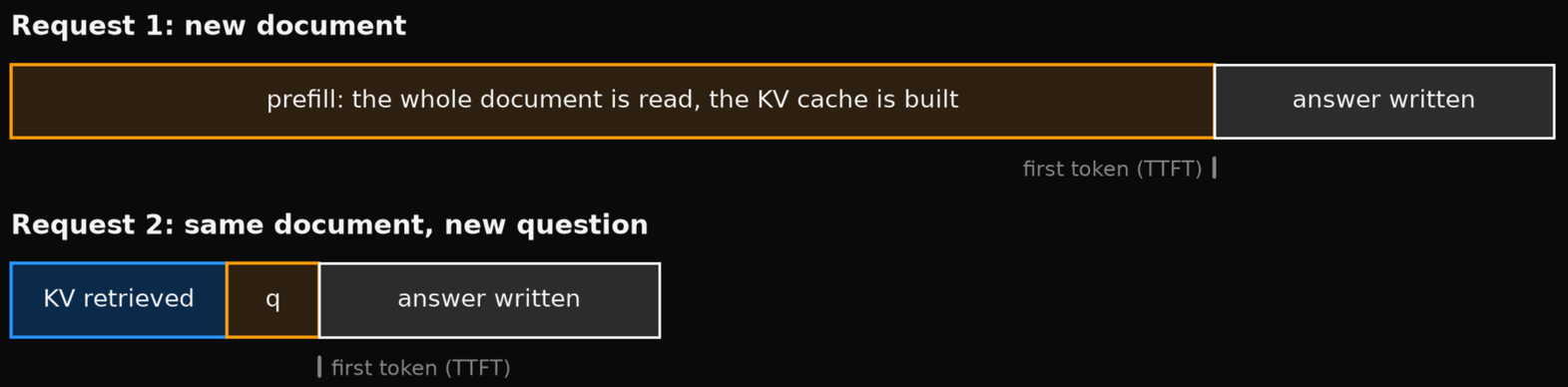



Figure 4. The long prefill is replaced by a short retrieve when the cached KV is reused. Measured here: 0.62 s cold vs 0.16 s warm (Qwen3-8B, 8,000-token documents, CXL L1).

KV cache: what is kept in memory by the model about text it has read

TTFT: time to first token. For long text, mostly prefill time.

Today's question: is time saved when the cached KV is retrieved instead of prefilled again?

THE BASICS

# Why the cache is outgrown

```
bytes/token = 2 × layers × kv_heads × head_dim × dtype_bytes
```

Bigger model, longer text, more users: more bytes.

**GPU**
**the busy work**

→

**DRAM**
**recent text**

→

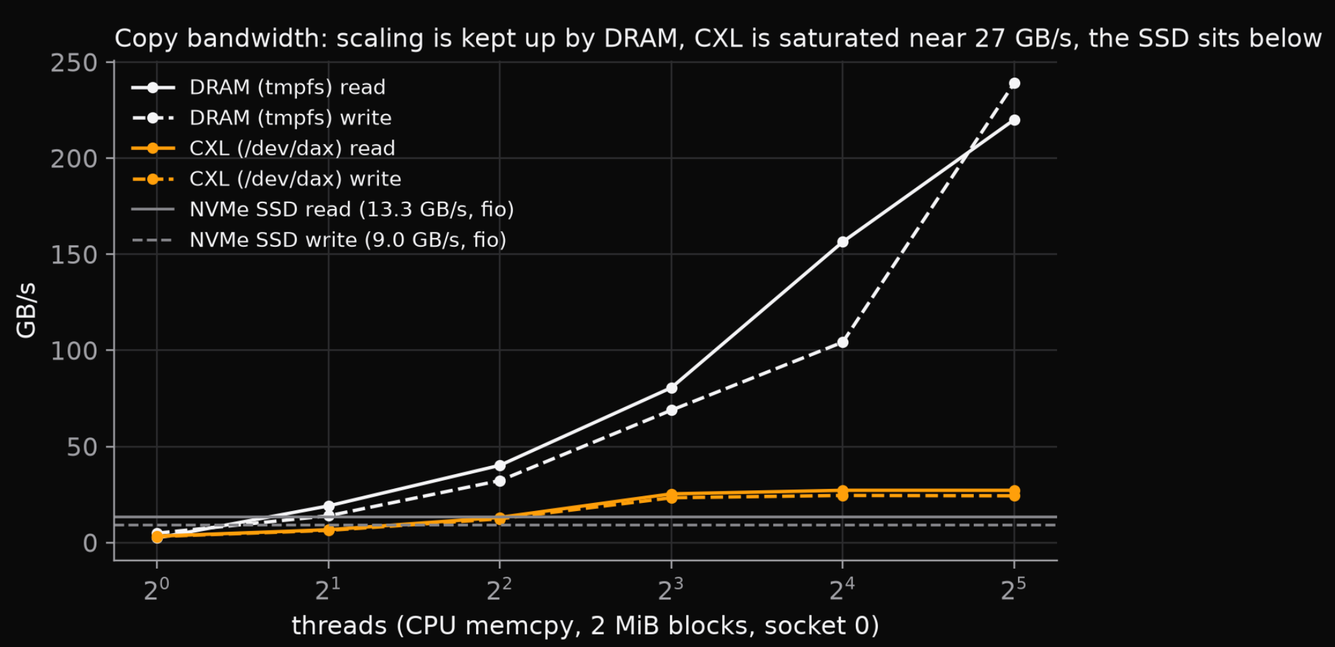
**disk or network**
**everything older**

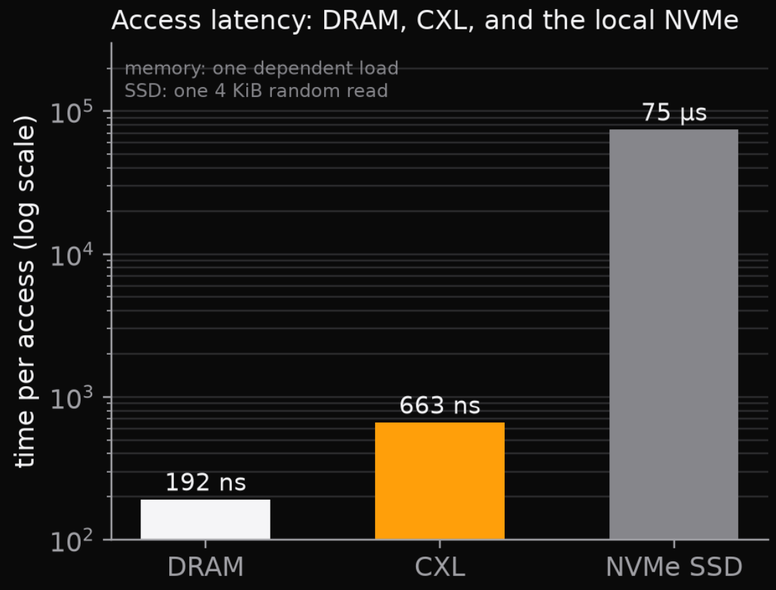
With **CXL**, more memory is added than the DRAM slots allow. It is used like memory, not like a disk. It can be shared by two servers.

Today: a Device-DAX L1, then a DAX L2, then a shared L1, built step by step.

THE BASICS

# DRAM, CXL and the local SSD



THE BASICS

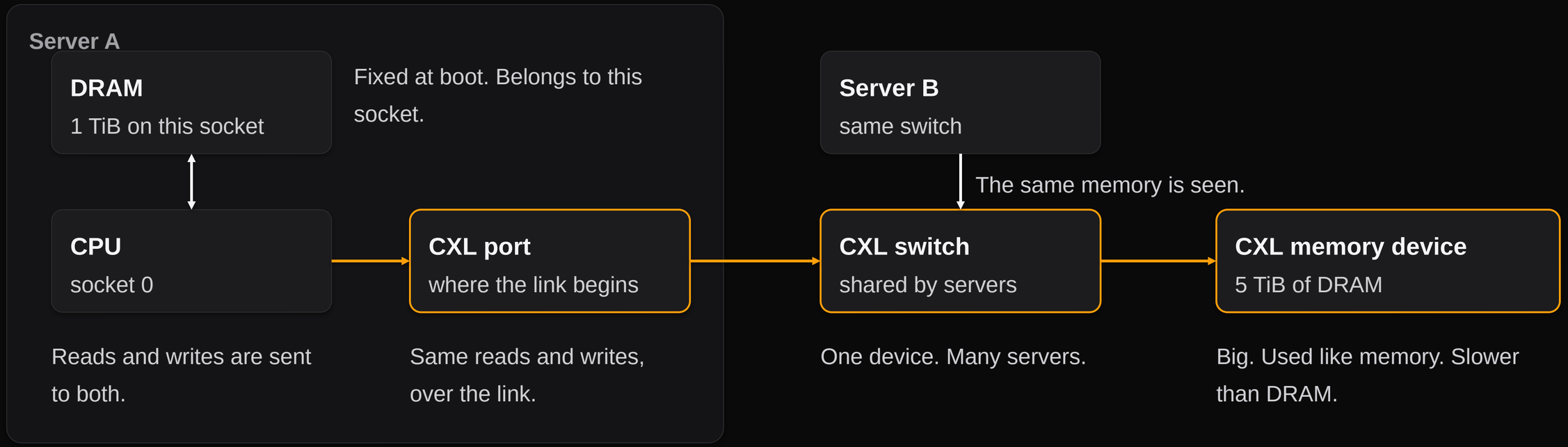
# Four things

| Word | What is meant here |
|---|---|
| **CXL(protocol) memory** | Extra memory that is connected over a CXL link instead of a DRAM slot. |
| **Device-DAX, devdax** | A Linux device file. That memory can be taken by one process and used directly. |
| **MP mode** | LMCache is run as its own process, next to the inference engine. Even on one server. |
| **L1 and L2** | LMCache’s tiers. The GPU is served by an L1 tier. An L2 adapter is the bigger store behind it. |

Next: the link, the Linux door, the two processes, then the tiers.

THE BASICS

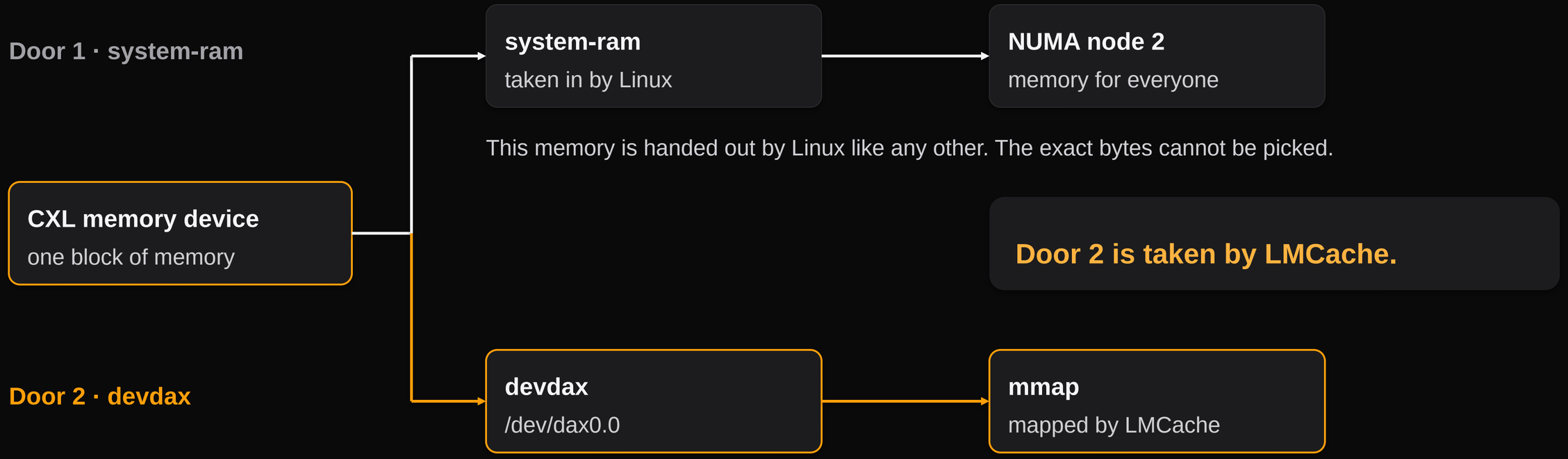
# Memory on a link




By the CPU it is seen as just more memory. The question is by whom it may be used.

THE BASICS

# Two ways in CXL memory.



THE BASICS

# Why devdax was chosen

| system-ram | devdax |
| --- | --- |
| **By whom the bytes are picked**<br>By Linux. The memory node can be picked, not the bytes. | **By whom the bytes are picked**<br>By the LMCache server. This device, this offset. |
| **By whom else it can be used**<br>By everyone. That is what system-ram is for. | **By whom else it can be used**<br>Only by the process that opens the device. |
| **Page size and lifetime**<br>Small pages, unless they are merged by Linux. With pinned pages the node is locked in. | **Page size and lifetime**<br>Big pages, pinned once. Memory can be added or removed while running. |
| **Two servers, one switch**<br>The same memory is believed free by two Linux kernels. Data is corrupted. | **Two servers, one switch**<br>The same offset is the same bytes on both servers. |

THE BASICS

# MP mode: two processes

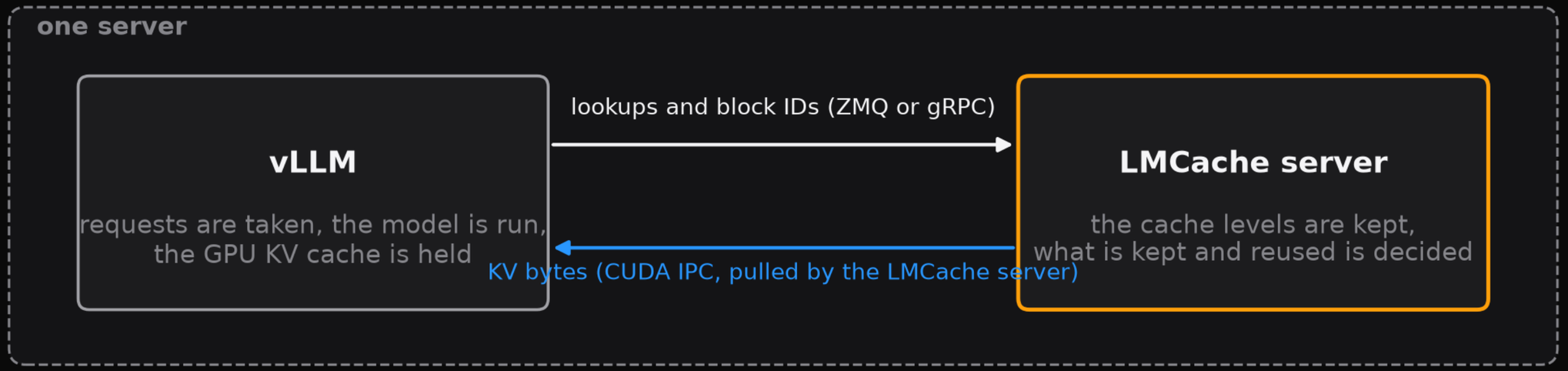



Figure 5. MP mode: the inference engine and the LMCache server are separate processes, even on one server.

- The inference engine is kept simple
- The cache is kept alive if the engine is restarted
- A second server? Not until step 7

THE BASICS

# Same hardware, three jobs

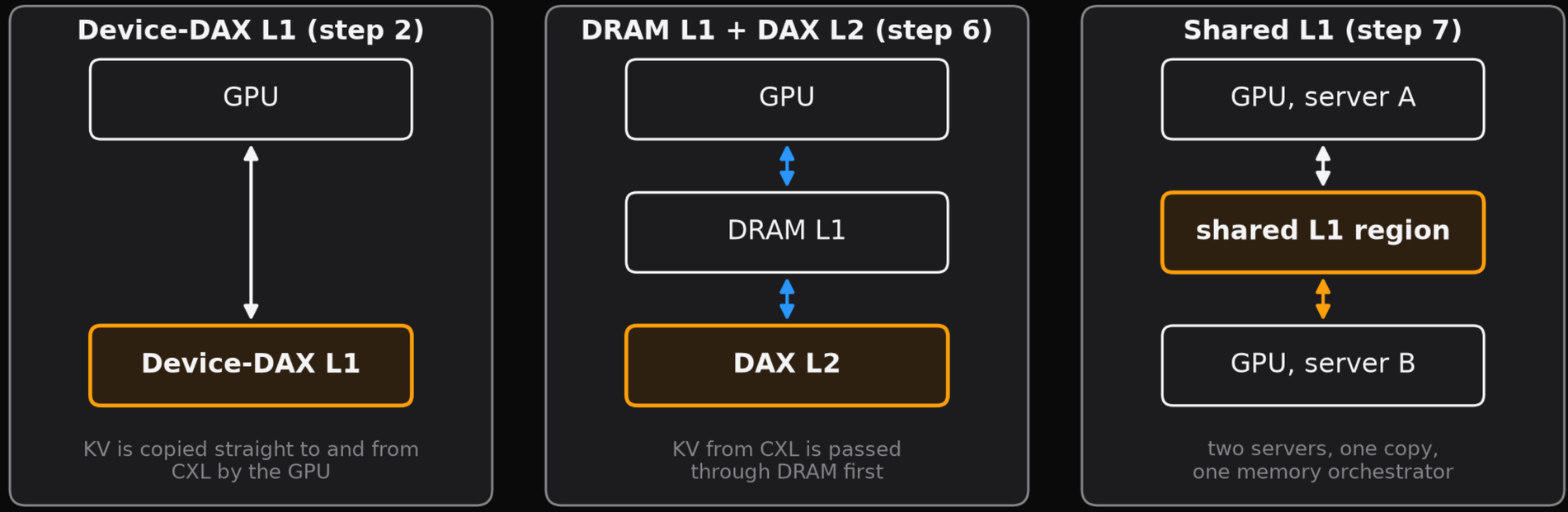



Figure 2. The same CXL memory in three software roles. Highlighted boxes are the CXL-backed level.

These three are built in the deployment chapter, in that order: steps 2, 6 and 7. The flags are shown on the next slide.

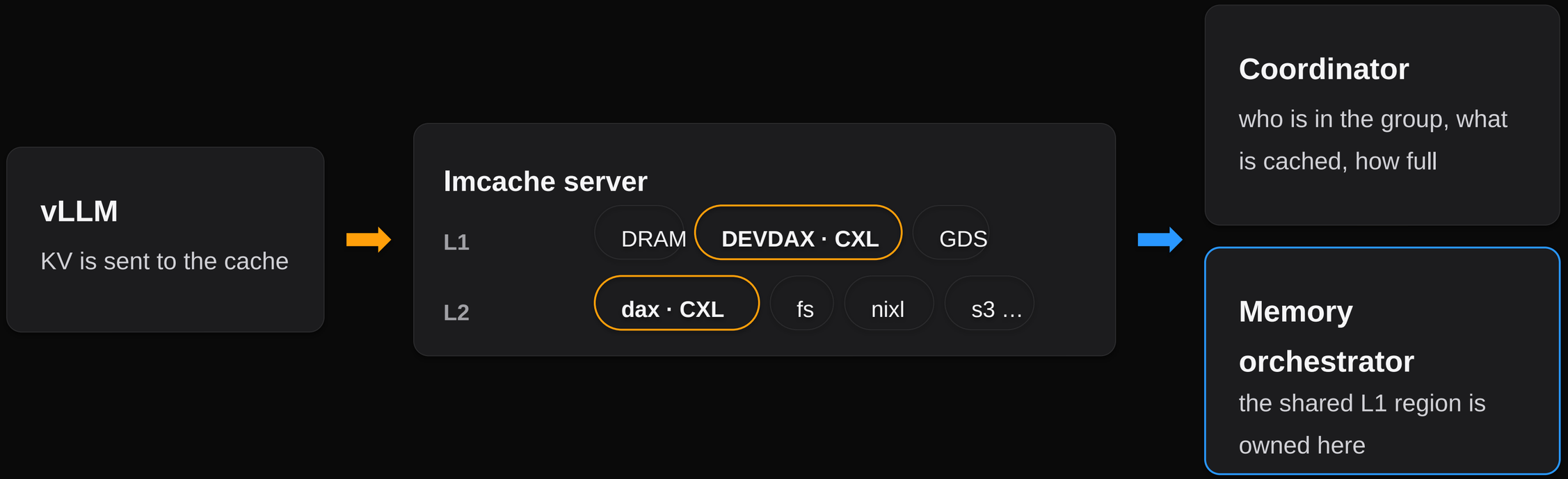
THE BASICS

# Three ways it can be set up

| | Device-DAX L1 | DRAM L1 + DAX L2 | Shared L1 (Upstream soon) |
|---|---|---|---|
| **Flags** | --l1-manager, type DEVDAX | --l2-adapter, type dax | a shared region, plus lmcache memory |
| **Data path** | Copied to CXL by the GPU directly | Copied into fixed slots by the CPU | Copied by the GPU, with visibility fences |
| **Scope** | One server | One server | Many servers |
| **While running** | Added, drained | Added, resized, removed | Fixed. Reset when stopped. |

THE BASICS

# Where CXL is plugged in




CXL is found three times: as an L1, as an L2, and as the shared L1 region.

THE BASICS

# The machines

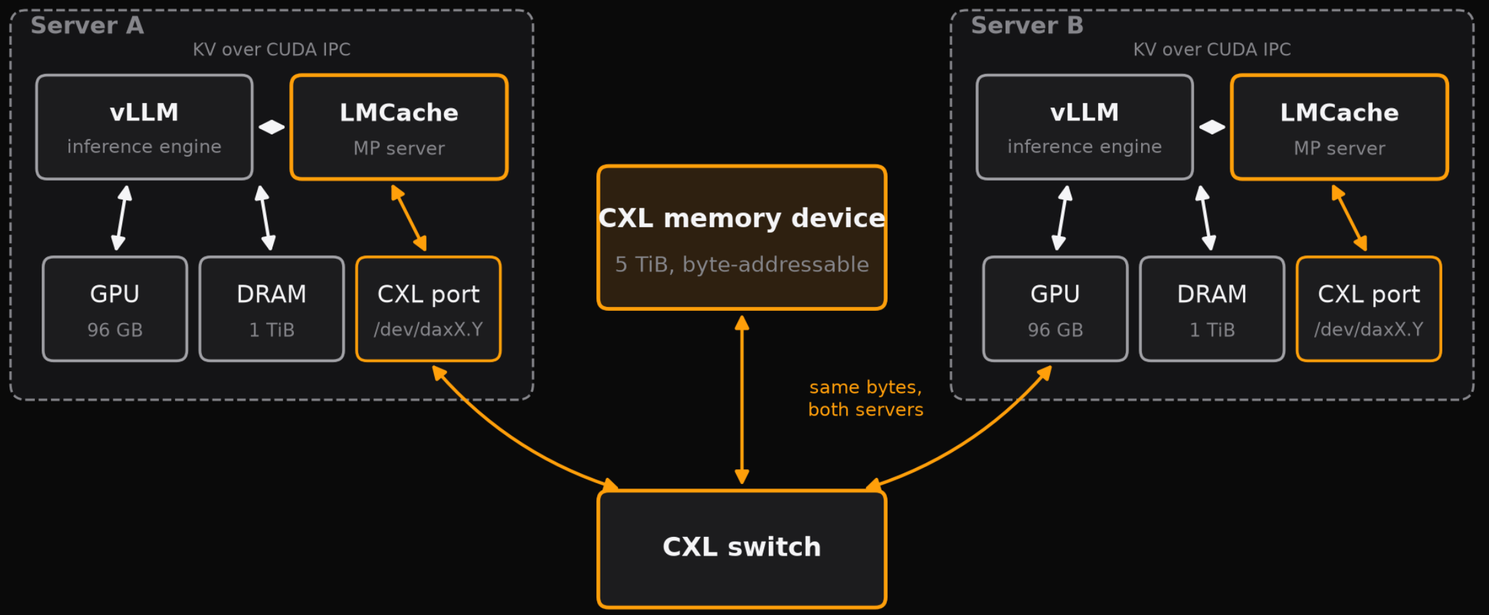



Figure 1. Two servers, one CXL memory device behind a switch. vLLM and an LMCache server are run on each server.

2 sockets · 8 GPUs with 96 GB · 2 TiB DRAM per server

2 × 5 TiB CXL memory, one device per socket, seen by both servers

THE BASICS

# From the device to /dev/dax

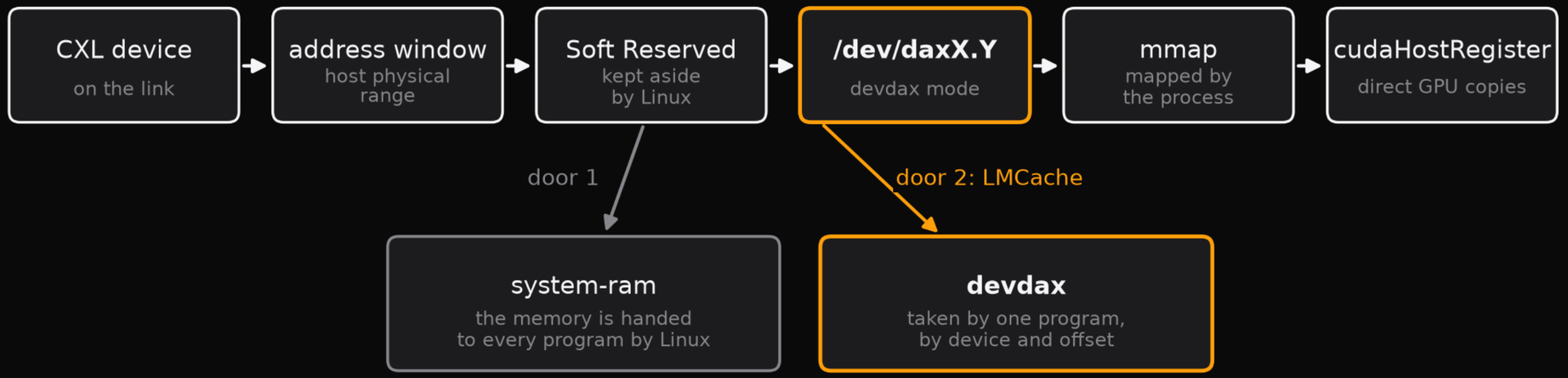



Figure 3. How a program on Linux is reached by CXL memory. Door 2 is used in the workshop.

Mapped at the device’s alignment: 2 MiB on this machine

Data is kept after the server exits. No index, nothing after power loss.

This memory is kept out of Linux’s general pool by devdax

THE BASICS

# Five steps from device to GPU

**Opened**
/dev/dax0.0

Size, alignment and socket are in sysfs.

**Mapped**
mapped into the process

The offset is picked by the LMCache server. The same offset can be picked on server B.

**Page tables**
pointed at the device

No extra copies are kept by Linux, and it is never swapped out.

**CUDA is told**
cudaHostRegister, once

Pinned once. It is never moved while the program runs.

**Copied**
GPU to CXL, directly

One copy per chunk. CPU copies are used only for the DAX L2 slots.

**Through system-ram**

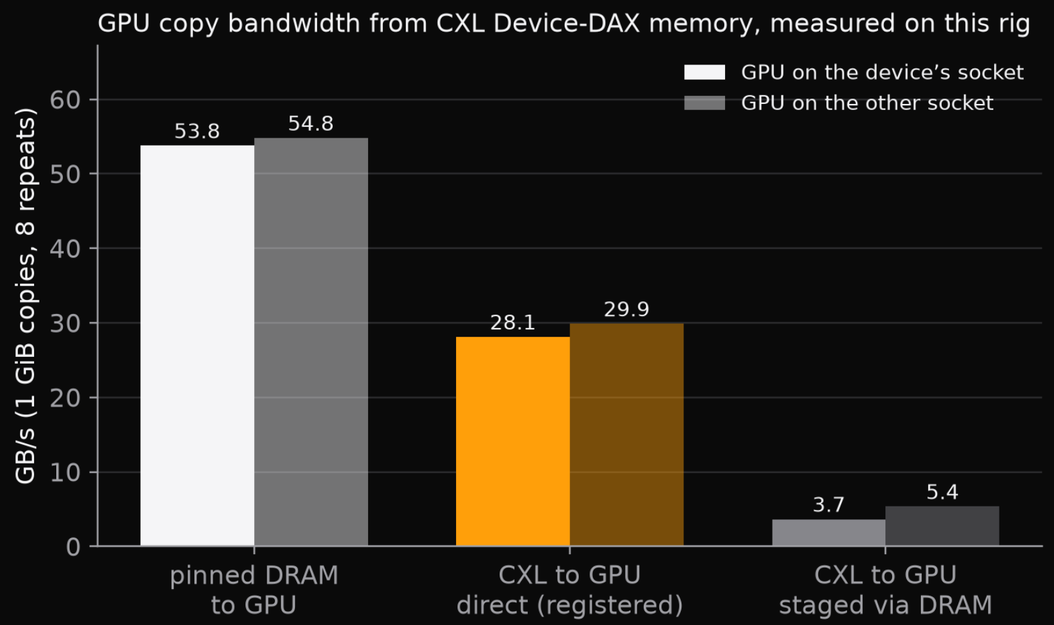
Memory chosen by Linux is given. Small pages unless they are merged by Linux. It can be moved, or lent to others.

**Through devdax**

A fixed block at a known spot on a known socket is given, pinned once. The same bytes can be named by two servers.

THE BASICS

# Is it seen by the GPU?




- The mapping is registered once with cudaHostRegister
- Then the GPU copies to and from the device directly
- Direct is 8× faster than the two-copy fallback here

Measured with the workshop probe: eight 1 GiB copies. The Device-DAX L1 does this registration at startup.

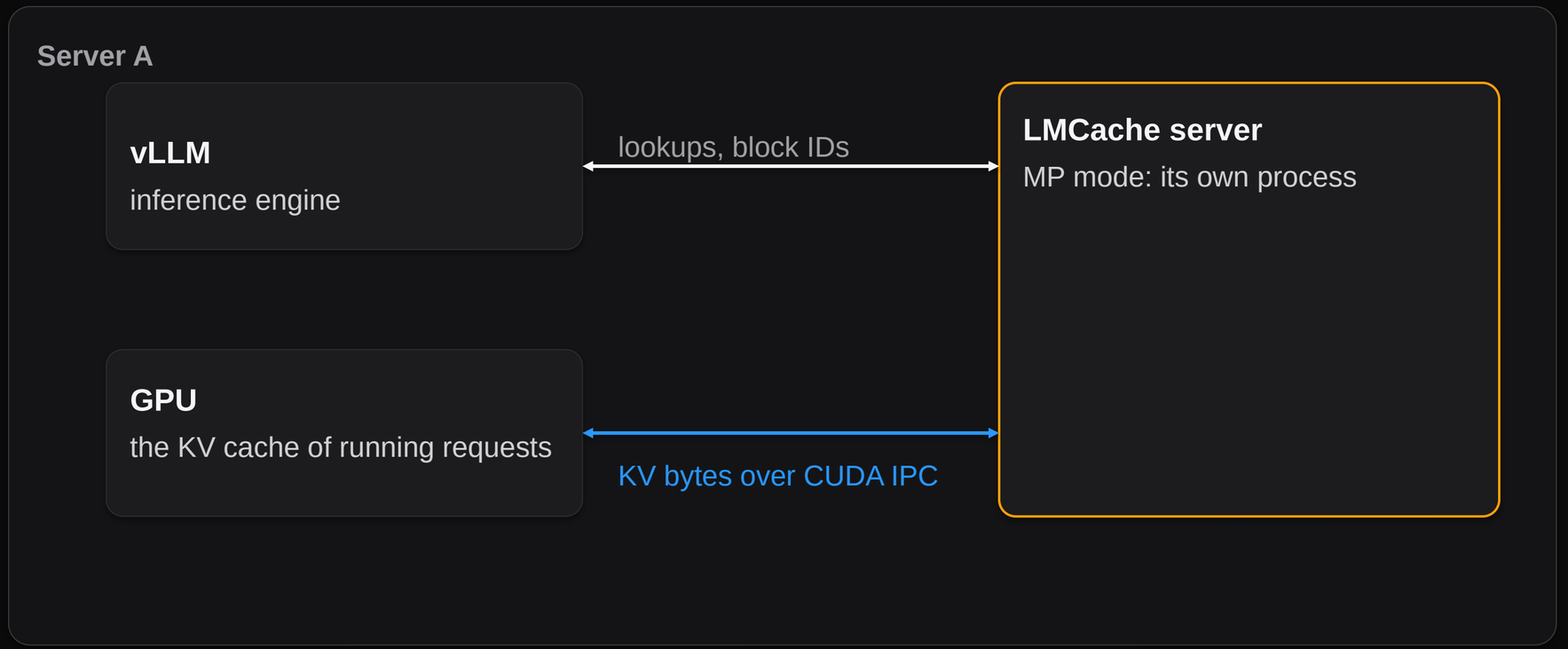
CHAPTER 2

# Deployment, step by step

One server. The Device-DAX L1. Hot-plug. The DAX L2. Then two servers on one shared L1.

DEPLOYMENT, STEP BY STEP

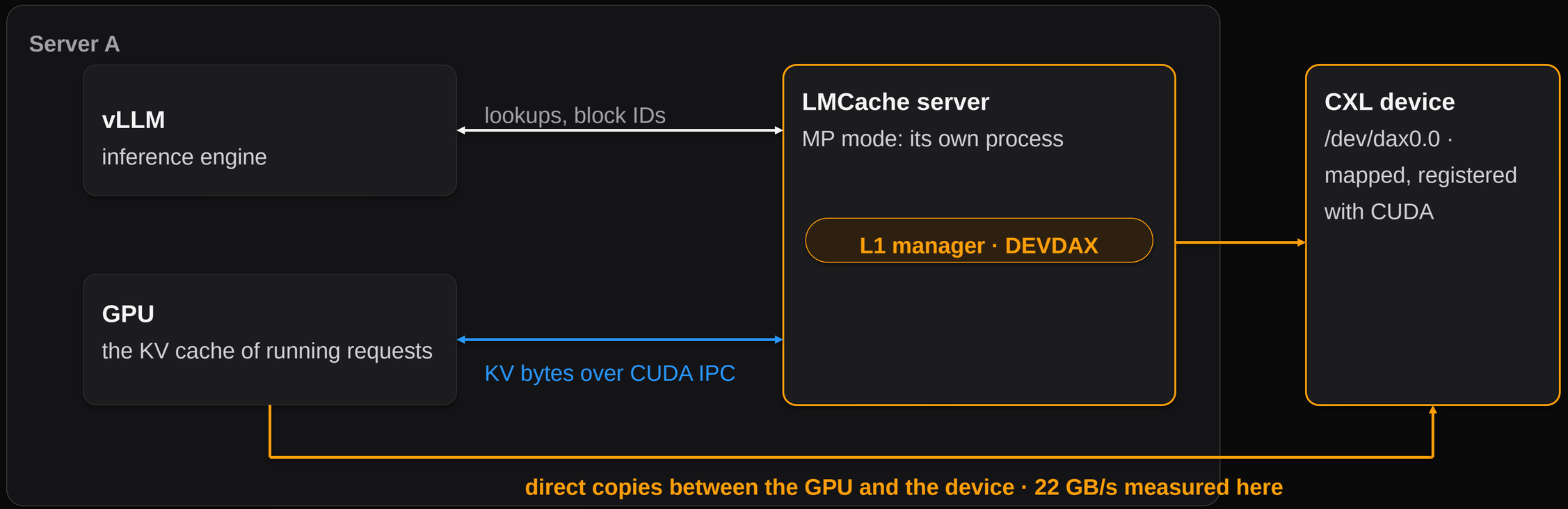
# One server, two processes




The engine is kept simple. The cache is run as its own process, and it survives an engine restart.

DEPLOYMENT, STEP BY STEP

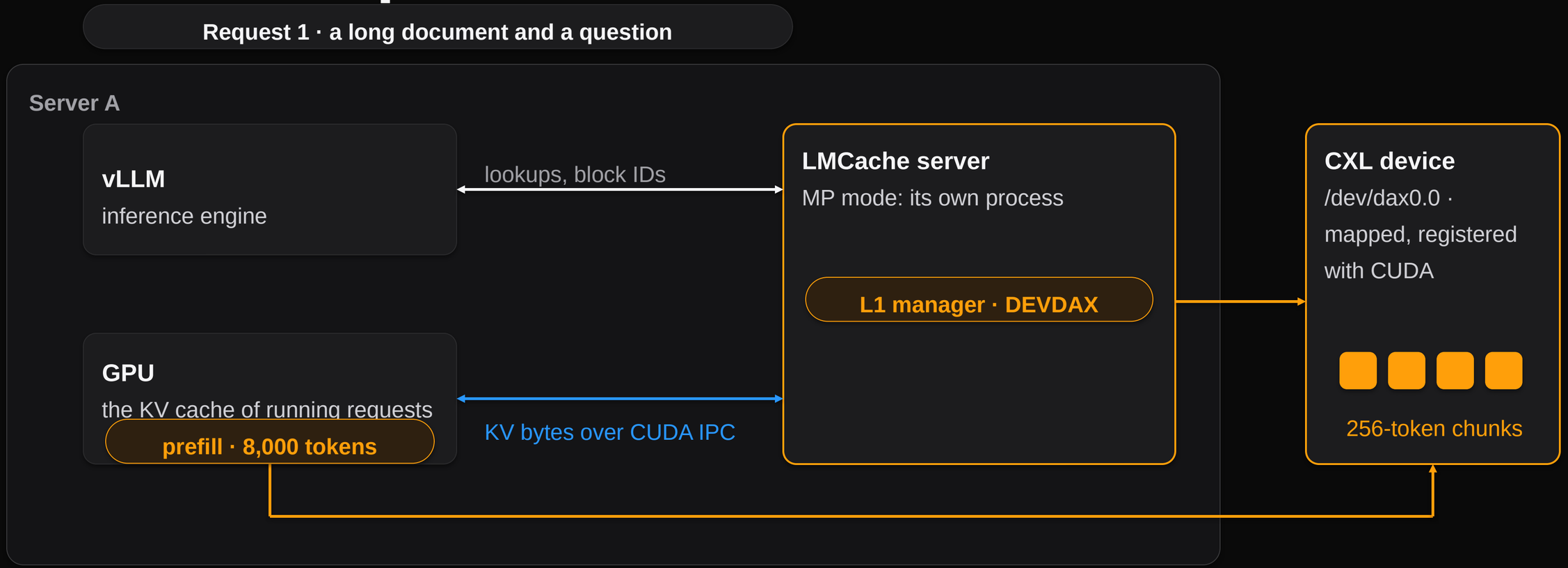
# The Device-DAX L1




The L1 is placed in CXL memory. Registered once with CUDA, the device is copied to and from by the GPU directly.

DEPLOYMENT, STEP BY STEP

# The first request



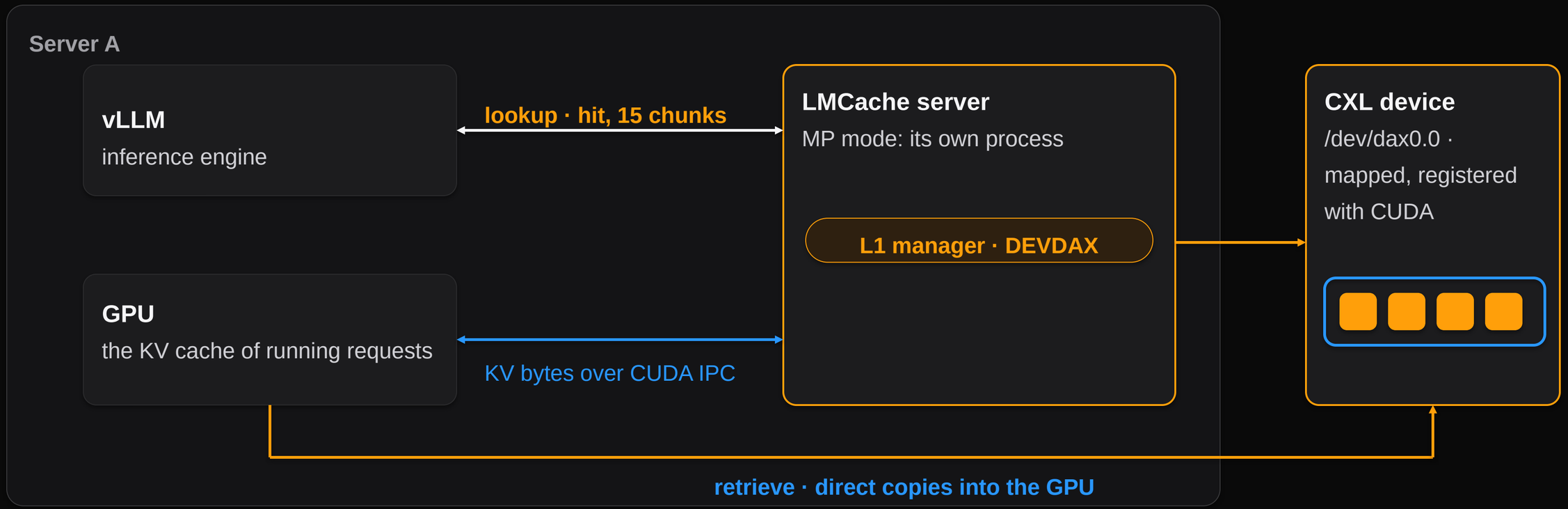

The document is prefilled on the GPU. Its KV is stored into the L1 in the background, chunk by chunk.

DEPLOYMENT, STEP BY STEP

# The second request

Request 2 · the same document, a new question

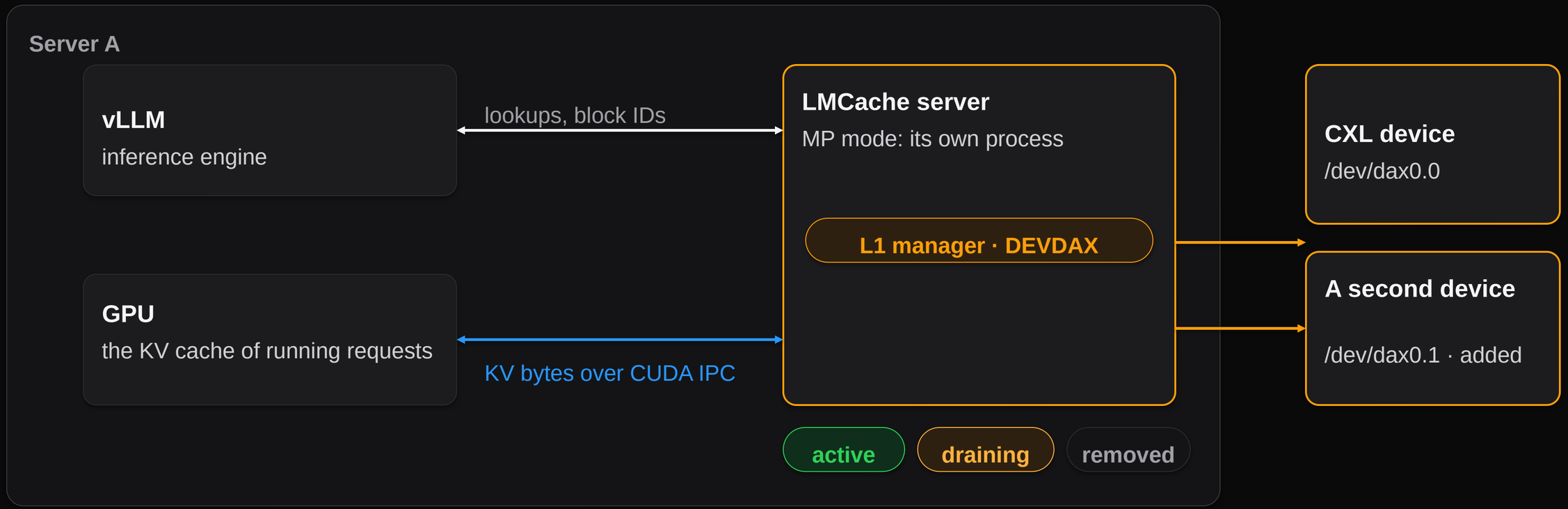
TTFT 0.62 s → 0.16 s, measured on this rig




Only the question is prefilled. The document’s KV is retrieved from CXL.

DEPLOYMENT, STEP BY STEP

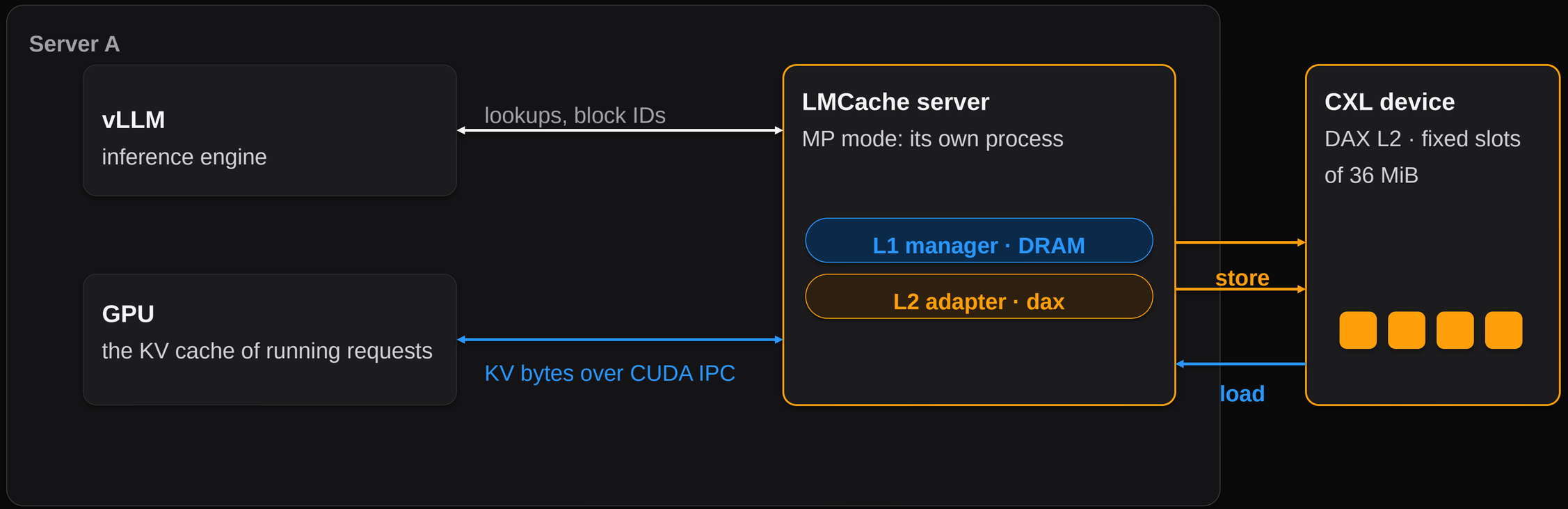
# Hot-plug




Capacity is added and drained while serving. Reads are still served while a device drains. No restart.

DEPLOYMENT, STEP BY STEP

# DRAM in front, CXL behind




Hot data stays in the DRAM L1. CXL becomes the bigger L2 behind it: stores are CPU copies into slots, loads are prefetched back.

DEPLOYMENT, STEP BY STEP

# Two jobs: ownership and visibility

### Ownership: the memory orchestrator

Where is a chunk kept? By whom may it be written? When may it be read by the other server? These are answered by one process, with small messages.

### Visibility: the fences

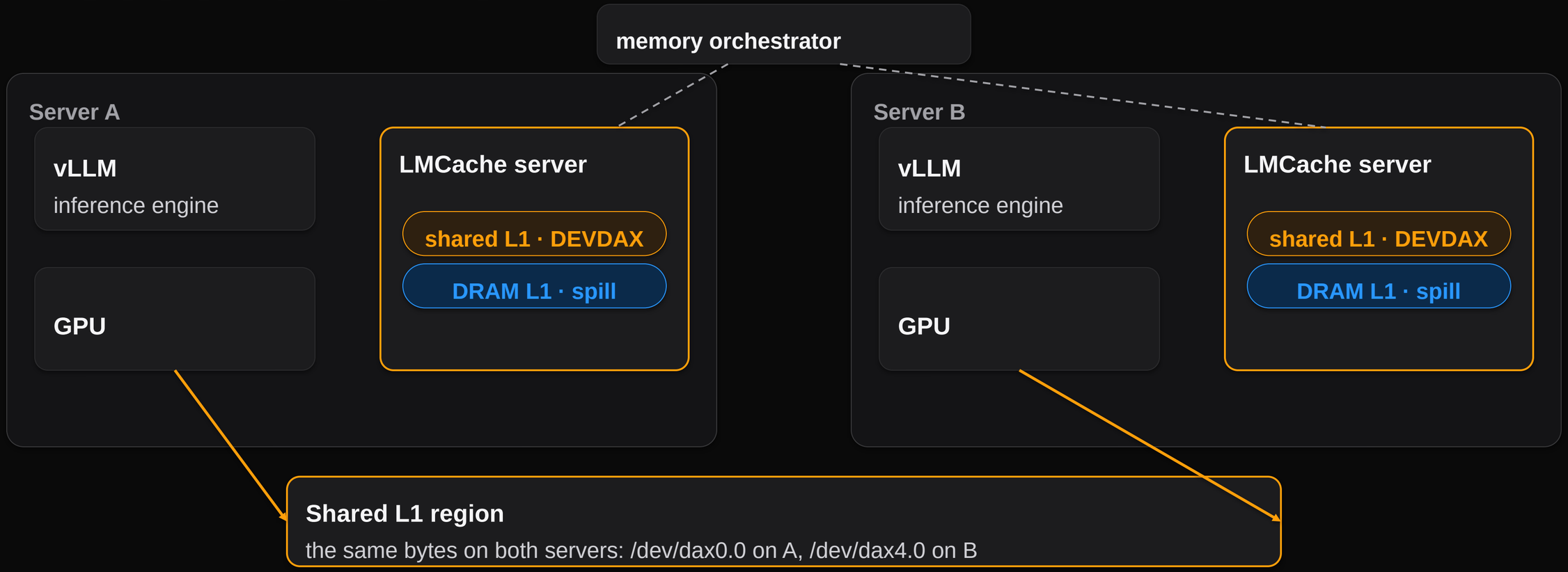
An old copy of the same memory may still be held by server B in its CPU cache. The new data is made visible by a publish fence on A and an acquire fence on B.

**A chunk is written by A. It is held in A’s CPU cache.** → **A read is made by B. The old data is still held in its own cache: stale.** → **A publish fence on A: CLFLUSHOPT, then SFENCE. The memory is reached by the data.** → **An acquire fence on B, then a read: fresh data.**

Ownership is covered on the next two slides. The exact fence steps come after.

DEPLOYMENT, STEP BY STEP

# A second server



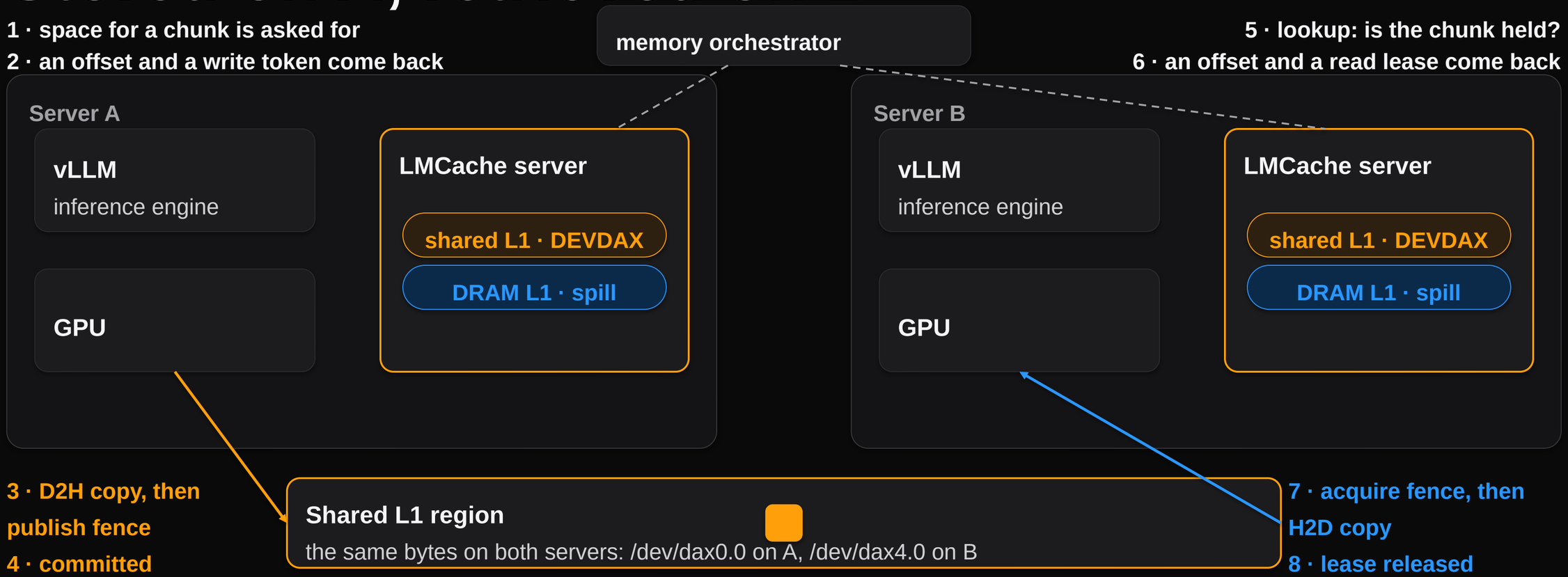

One region is mapped by both servers. One orchestrator keeps the bookkeeping. The KV bytes never pass through it.

DEPLOYMENT, STEP BY STEP

# Stored on A, retrieved on B

1 · space for a chunk is asked for
2 · an offset and a write token come back

5 · lookup: is the chunk held?
6 · an offset and a read lease come back




Eight small messages. The bytes are moved only once, through the shared region.

DEPLOYMENT, STEP BY STEP

# Why every copy is fenced

The two servers' CPU caches are not kept in sync by the CXL switch in this workshop, so it is done by software fences (software_fenced). With newer CXL (3.0) it can be done in hardware: the setup at hand should be checked before this design is copied.

**Writer**

**GPU copy done** → **publish fence: CLFLUSHOPT, SFENCE** → **marked ready**

**Reader**

**a read is asked for** → **acquire fence on the range** → **copied to the GPU**

⚠ coherent mode: only for one server, or hardware by which caches are kept in sync

✓ Chunks are 2 MiB aligned, so a cache line is never shared by two writers